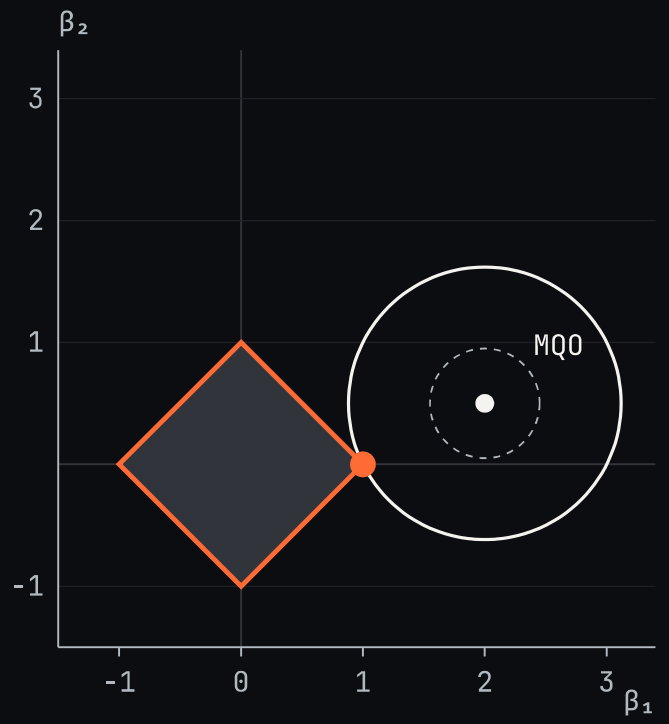
AULA 7.1 — 2 HORAS

# Regulari-zação

## Ridge, Lasso e ElasticNet




Como controlar a complexidade de um modelo?

# Onde esta aula se encaixa

ANTES

## Regressão linear, treino e teste

Minimizar o erro sem limitar os coeficientes

HOJE — 7.1

## Erro + penalidade sobre os coeficientes

Ridge (L2), Lasso (L1), ElasticNet (L1 + L2)

DEPOIS — 7.2

## SVM: margem máxima

Minimizar $\|w\|^2$ é uma penalidade L2

# Objetivos

01 Reconhecer overfitting e underfitting nas curvas de erro

02 Escrever e explicar as funções objetivo de Ridge, Lasso e ElasticNet

03 Explicar pela geometria por que L1 zera coeficientes e L2 não

04 Escolher alpha por validação cruzada dentro de um Pipeline

05 Interpretar coeficientes e o caminho de regularização

# Roteiro de 120 minutos



00–50 min

## Teoria

Recapitulação, complexidade, Ridge, Lasso, geometria, ElasticNet

50–60

## Pausa

60–85 min

## Alpha e validação

Caminho, escala, CV, quiz

85–120 min

## Exercício guiado

Notebook do zero, erros comuns, desafio

# Regressão linear

$$\hat{y}_i = \beta_0 + \Sigma_{j=1..p} \beta_j x_{ij}$$

$\hat{y}_i$
previsão para a observação i

$\beta_0$
intercepto, previsão quando todo x vale 0

$\beta_j$
coeficiente: efeito de +1 unidade em $x_j$

$x_{ij}$, $p$
valor da variável j na observação i; p variáveis

# Função custo dos mínimos quadrados

$$RSS(\beta) = \Sigma_{i=1..n} (y_i - \hat{y}_i)^2$$

$$MSE = RSS / n \quad RMSE = \sqrt{MSE}$$

---

$y_i - \hat{y}_i$
resíduo: valor real menos valor previsto

$(\cdot)^2$
evita cancelamento e pune erros grandes

$n$
número de observações de treino

# Três pontos, resolvidos à mão

| x | y | x - $\bar{x}$ | y - $\bar{y}$ |
|---|---|---|---|
| 1 | 2 | -1 | -4/3 |
| 2 | 3 | 0 | -1/3 |
| 3 | 5 | 1 | 5/3 |

$\bar{x} = 2$  $\bar{y} = 10/3$

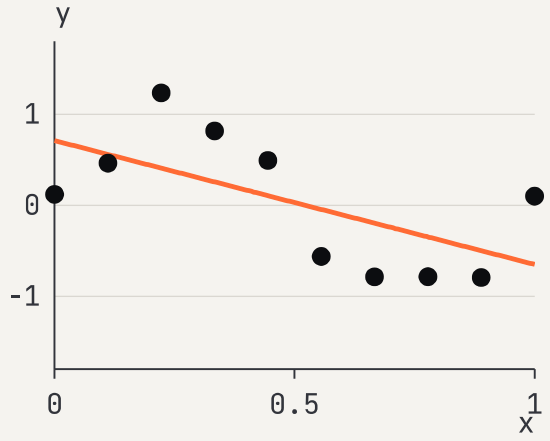
| | |
|---|---|
| $S_{xy} = \Sigma(x-\bar{x})(y-\bar{y})$ | 3 |
| $S_{xx} = \Sigma(x-\bar{x})^2$ | 2 |
| $\beta_1 = S_{xy} / S_{xx}$ | 1,5 |
| $\beta_0 = \bar{y} - \beta_1\bar{x}$ | 1/3 |
| $RSS$ | 1/6 ≈ 0,167 |

01

CAPÍTULO 01

# O problema da complexidade

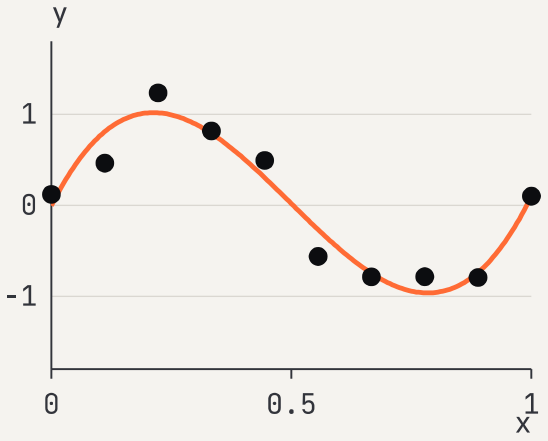
# Underfitting e overfitting




grau 1

**Simples demais**

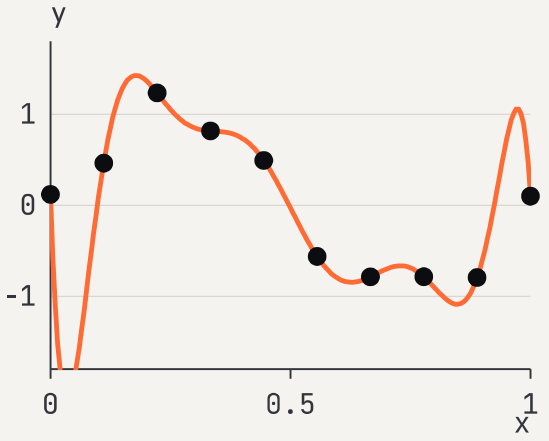
Erra no treino e em dados novos




grau 3

**Acompanha a tendência**

Ignora a maior parte do ruído




grau 9

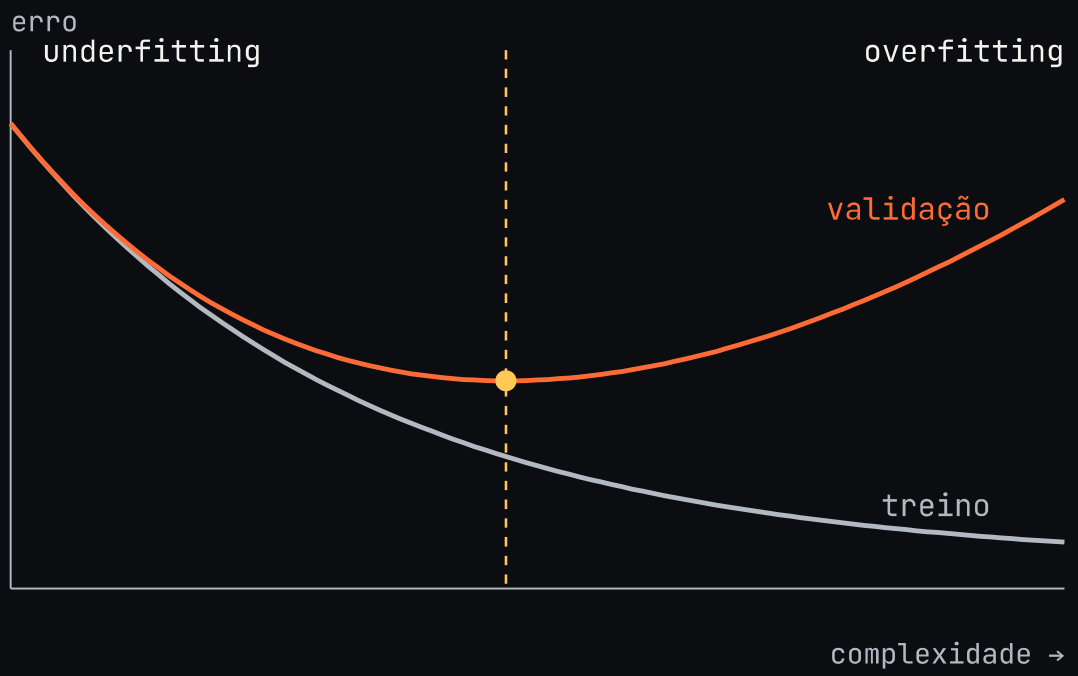
**Passa por todos os pontos**

Erro de treino zero, oscila entre os pontos

# Erro de treino e erro de validação

O erro de treino sempre cai com a complexidade. O erro de validação cai e depois sobe.

Esquema qualitativo



# Erro mais penalidade

$$J(\beta) = erro + \alpha \cdot penalidade(\beta)$$

| $\alpha = 0$ | $\alpha$ moderado | $\alpha \to \infty$ |
| --- | --- | --- |
| Coeficientes gratuitos: volta ao MQO | Só variáveis úteis recebem coeficientes grandes | Coeficientes → 0; prevê quase a média de y |

02

CAPÍTULO 02

# Ridge: penalidade L2

# Função objetivo do Ridge

$$J_{Ridge}(\beta) = \Sigma_i (y_i - \hat{y}_i)^2 + \alpha \, \Sigma_{j=1..p} \, \beta_j^2$$

$$= RSS(\beta) + \alpha \, \|\beta\|_2^2$$

---

$RSS(\beta)$
ajuste aos dados de treino

$\alpha \geq 0$
força da penalidade; hiperparâmetro

$\|\beta\|_2^2$
soma dos quadrados dos coeficientes

$j = 1..p$
$\beta_0$, o intercepto, não é penalizado

# Normas L1 e L2

L1

$$\|\beta\|_1 = \Sigma\,|\beta_j|$$

soma dos valores absolutos

L2

$$\|\beta\|_2 = \sqrt{\Sigma\,\beta_j^2}$$

comprimento do vetor

$L2^2$

$$\|\beta\|_2^2 = \Sigma\,\beta_j^2$$

usada no Ridge

---

$\beta = (3, -4)$ L1 = 7 L2 = 5 $L2^2$ = 25

# Ridge nos três pontos

$$\beta_1 = S_{xy} / (S_{xx} + \alpha)$$

Derivada de $\Sigma(y-\bar{y} - \beta_1(x-\bar{x}))^2 + \alpha\beta_1^2$ igualada a zero.
Com $S_{xy} = 3$ e $S_{xx} = 2$:

| α | conta | $\beta_1$ |
|---|---|---|
| 0 | 3/2 | 1,500 |
| 2 | 3/4 | 0,750 |
| 10 | 3/12 | 0,250 |
| 100 | 3/102 | 0,029 |

Encolhe continuamente e só chega a zero quando $\alpha \to \infty$.

# Solução do Ridge em forma matricial

$$\hat{\beta}_{MQO} = (X^T X)^{-1} X^T y$$

$$\hat{\beta}_{Ridge} = (X^T X + \alpha I)^{-1} X^T y$$

$X$
n × p, variáveis centralizadas

$y$
vetor com os n alvos

$\alpha I$
soma α à diagonal; garante inversa se α > 0

$(\cdot)^{-1}$
matriz inversa

03

CAPÍTULO 03

# Lasso: penalidade L1

# Função objetivo do Lasso

$$J_{Lasso}(\beta) = 1/2n \, \Sigma_i (y_i - \hat{y}_i)^2 + \alpha \, \Sigma_{j=1..p} |\beta_j|$$

$$= RSS(\beta)/2n + \alpha \, \|\beta\|_1$$

---

**$1/2n$**

erro médio; o ½ simplifica a derivada

**$|\beta_j|$**

valor absoluto: cresce em linha reta

**$\alpha$**

não é comparável ao α do Ridge

**`solver`**

descida por coordenadas, iterativo

# Lasso nos três pontos

$$\beta_1 = max(S_{xy} - \alpha, 0) / S_{xx}$$

Mínimo de $\frac{1}{2}\Sigma(y-\bar{y} - \beta_1(x-\bar{x}))^2 + \alpha|\beta_1|$ para $S_{xy} > 0$. Chama-se limiar suave (soft thresholding).

| α | conta | $\beta_1$ |
|---|---|---|
| 0 | 3/2 | 1,5 |
| 1 | 2/2 | 1,0 |
| 2 | 1/2 | 0,5 |
| 3 | 0/2 | 0 |
| 5 | corte | 0 |

Para $\alpha \geq S_{xy} = 3$, o coeficiente é exatamente zero.

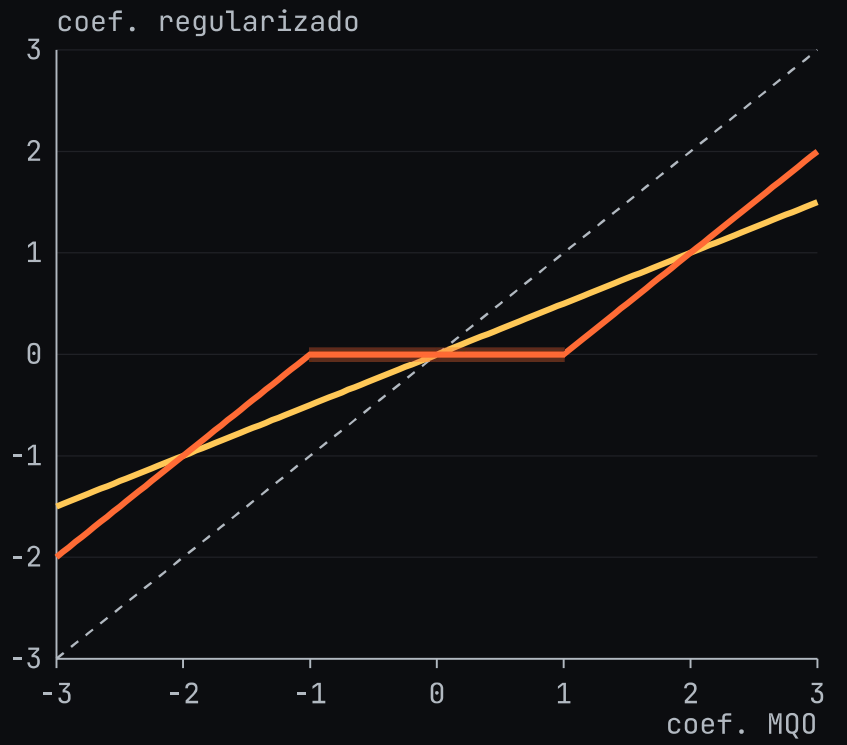
# Como cada penalidade encolhe um coeficiente

Caso exato: $X^TX = I$, $\alpha = 1$

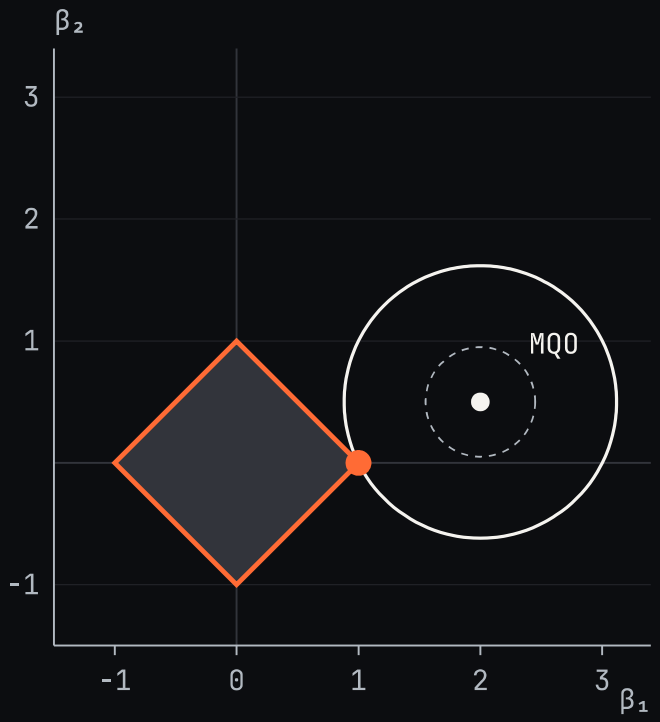
----- MQO: $\beta = \hat{\beta}$

— Ridge: $\beta = \hat{\beta} / (1 + \alpha)$

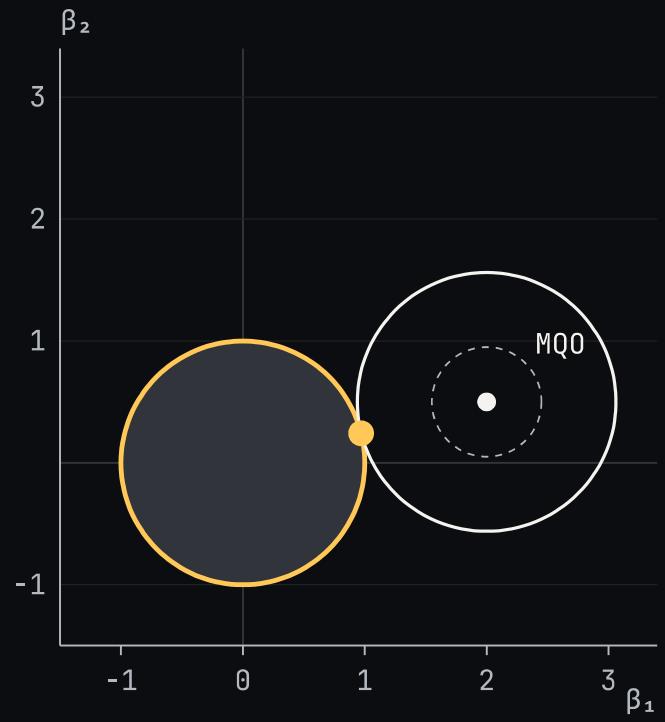
— Lasso: $\beta = \text{sinal}(\hat{\beta}) \cdot \max(|\hat{\beta}| - \alpha, 0)$



# Geometria das regiões de penalização




L1: $|\beta_1| + |\beta_2| \leq 1 \rightarrow (1;\ 0)$




L2: $\beta_1^2 + \beta_2^2 \leq 1 \rightarrow (0{,}97;\ 0{,}24)$

O erro mínimo dentro da região é o primeiro ponto tocado pelas curvas de erro.

---

$\hat{\beta} = (2;\ 0{,}5)$. Curvas de erro circulares porque $X^TX = I$.

# Por que L1 zera: a derivada

PASSO 1 — L2

$$d(\alpha\beta^2)/d\beta = 2\alpha\beta$$

O empurrão enfraquece quando β se aproxima de zero.

PASSO 2 — L1

$$d(\alpha|\beta|)/d\beta = \pm\alpha$$

O empurrão tem a mesma força em qualquer β ≠ 0.

PASSO 3 — EM ZERO

$$|S_{xy}| \leq \alpha \Rightarrow \beta = 0$$

Se o erro puxa com força menor que α, β fica em zero.

04

CAPÍTULO 04

# ElasticNet: L1 + L2

# Função objetivo do ElasticNet

$$J(\beta) = RSS/2n + \alpha\rho \|\beta\|_1 + \tfrac{1}{2}\alpha(1-\rho) \|\beta\|_2^2$$

| $\alpha$ | $\rho = l1_ratio \in [0, 1]$ | $1 - \rho$ |
|---|---|---|
| força total da penalidade | fração da força dada à L1 | fração dada à L2 |

`ρ = 1 → Lasso` `ρ = 0 → Ridge, outra escala de α` `correlacionadas → peso dividido`

# A mesma ideia, escalas diferentes

| IMPLEMENTAÇÃO | FUNÇÃO OBJETIVO |
|---|---|
| sklearn Ridge | $RSS + \alpha\|\beta\|_2^2$ |
| sklearn Lasso | $RSS/2n + \alpha\|\beta\|_1$ |
| sklearn ElasticNet | $RSS/2n + \alpha\rho\|\beta\|_1 + \frac{1}{2}\alpha(1-\rho)\|\beta\|_2^2$ |
| glmnet (R) | $RSS/2n + \lambda[\alpha\|\beta\|_1 + \frac{1}{2}(1-\alpha)\|\beta\|_2^2]$ |

No glmnet, α é a proporção (o l1_ratio do sklearn) e λ é a força. Compare modelos pelo erro de validação, nunca pelo valor de α.

# O intercepto não é penalizado

$\beta_0$ só posiciona o nível médio das previsões. Penalizá-lo puxaria tudo para zero e tornaria o modelo dependente da origem de y.

`fit_intercept=True`
padrão: estima $\beta_0$ sem penalidade

`modelo.intercept_`
valor de $\beta_0$ após o treino

`modelo.coef_`
$\beta_1 \dots \beta_p$, os coeficientes penalizados

$$X\ padronizado \Rightarrow \beta o = \bar{y}_{treino}$$

05

CAPÍTULO 05

# Alpha e o caminho de regularização

# Alpha controla a complexidade

$\alpha \to 0$

MQO. Baixo viés, alta variância.

$\alpha \to \infty$

$\beta \to 0$. Prevê a média de y.

$10^{-3}$ $10^{-2}$ $10^{-1}$ $10^{0}$ $10^{1}$ $10^{2}$ $10^{3}$

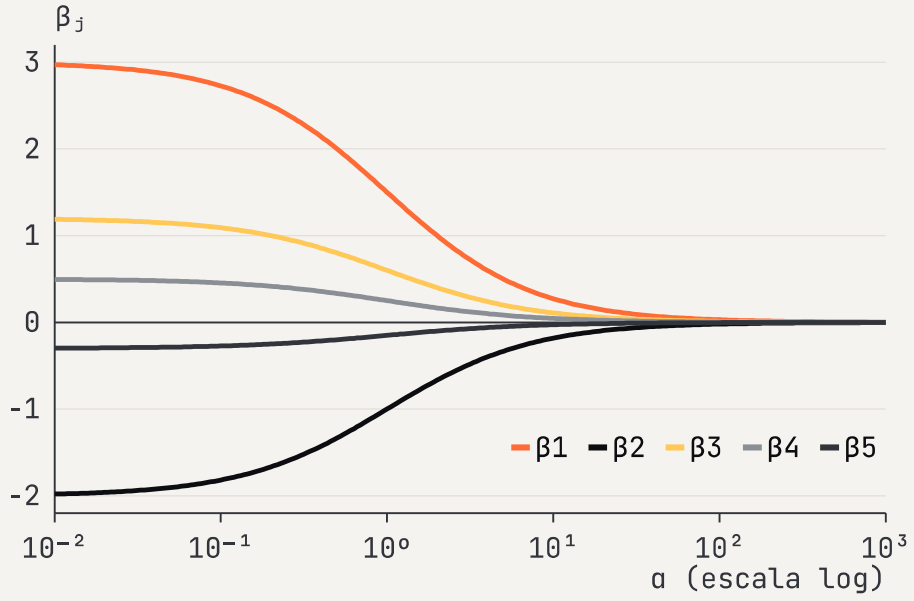
```
np.logspace(-3, 3, 7) # escala logarítmica
```

# Ridge: coeficientes quando alpha aumenta

$$\beta_j(\alpha) = \hat{\beta}_j / (1 + \alpha)$$

Todos encolhem juntos, de forma suave. Nenhum chega a zero.

Caso exato: $X^TX = I$; $\hat{\beta} = (3; -2; 1{,}2; 0{,}5; -0{,}3)$

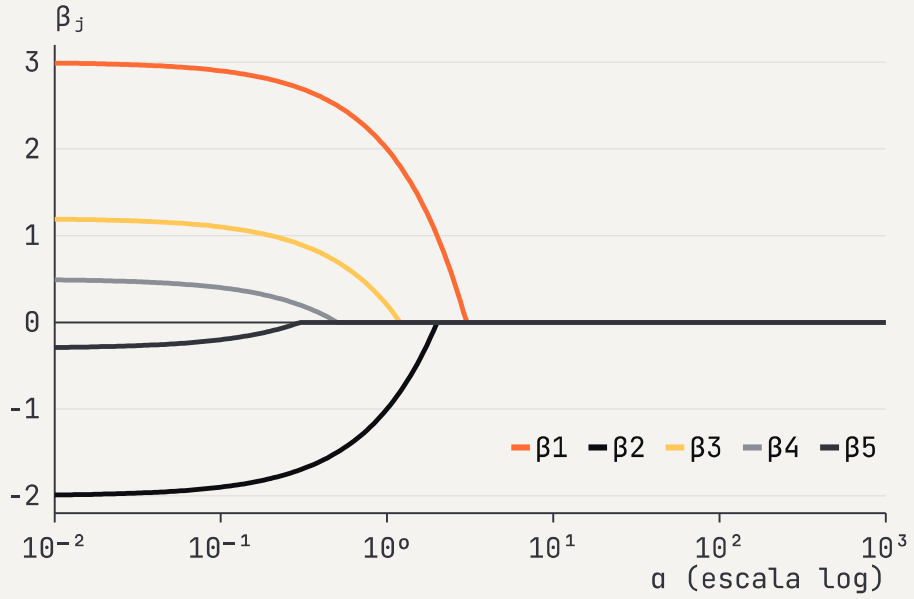


# Lasso: o caminho de regularização

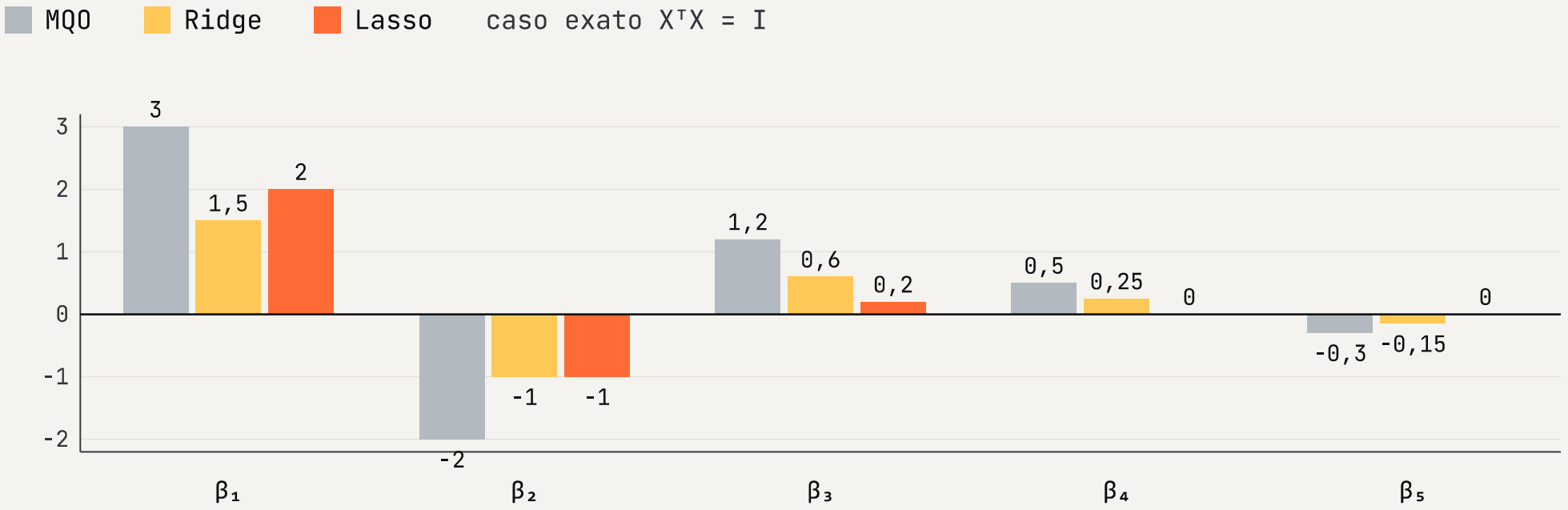
$$\beta_j(\alpha) = sinal(\hat{\beta}_j)\ max(|\hat{\beta}_j| - \alpha, 0)$$

Cada coeficiente chega a zero em $\alpha = |\hat{\beta}_j|$ e permanece lá.

Zeros em α = 0,3 · 0,5 · 1,2 · 2 · 3



# Coeficientes em α = 1




O Lasso zera $\beta_4$ e $\beta_5$ e encolhe menos os coeficientes grandes.

06

CAPÍTULO 06

# Escalonamento

# A penalidade depende da unidade

ÁREA EM $m^2$

**2 000**

R$ por $m^2$

$$\beta^2 = 4 \times 10^6$$

MESMA ÁREA EM $km^2$

**$2 \times 10^9$**

R$ por $km^2$

$$\beta^2 = 4 \times 10^{18}$$

Previsões idênticas, penalidades $10^{12}$ vezes diferentes. Padronize antes de regularizar.

# Padronização com StandardScaler

$$z_{ij} = (x_{ij} - \mu_j) / \sigma_j$$

(120 - 80) / 20 = 2

**fit(X_train)**
calcula μ e σ só com o treino

**transform(X)**
aplica os mesmos μ e σ a qualquer conjunto

**fit_transform**
os dois passos; apenas no treino

07

CAPÍTULO 07

# Validação cruzada

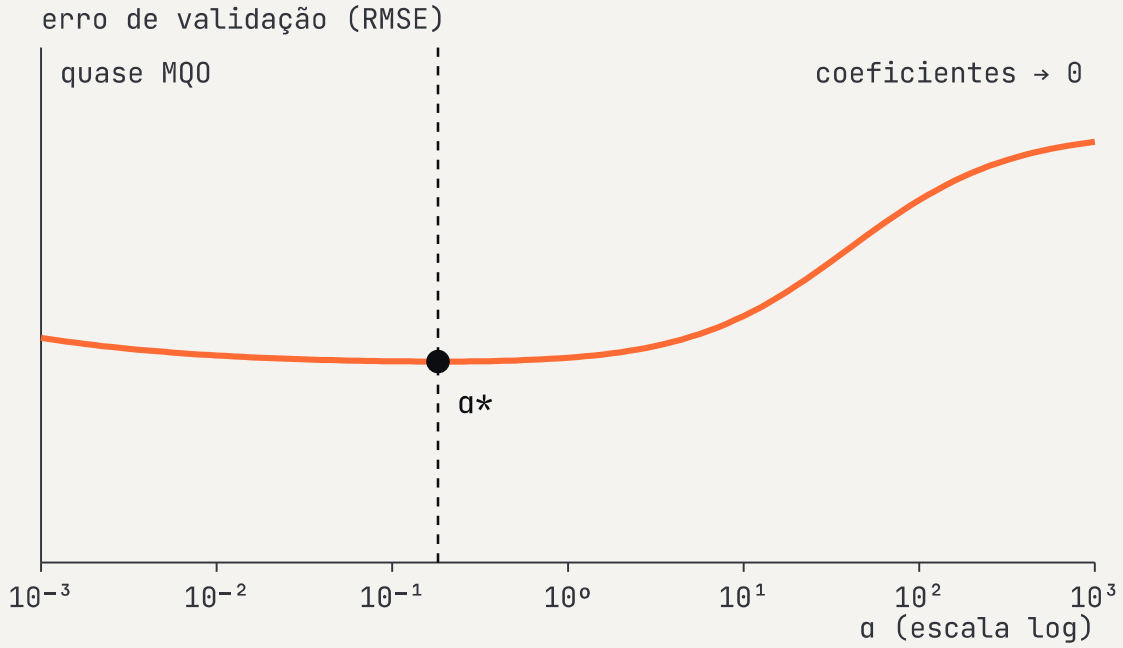
# Validação cruzada em 5 dobras



# Erro de validação em função de alpha

α* minimiza o erro médio nas dobras. À direita, o modelo se aproxima da média de y.

Esquema qualitativo. A curva real vem de cv_results_.



# Selecionar e avaliar são etapas distintas

TREINO — 80% | TESTE — 20%

SELECIONAR

**Validação cruzada escolhe α e o modelo**

Muitos candidatos, comparados nas mesmas dobras

AVALIAR

**Uma vez, no final**

# O Pipeline evita vazamento

ERRADO

```
Xs = StandardScaler().fit_transform(X_train)
cross_val_score(Ridge(), Xs, y_train, cv=cv)
```

μ e σ incluem a dobra de validação de cada rodada.

CORRETO

```
pipe = Pipeline([("scaler", StandardScaler()),
                 ("model", Ridge())])
cross_val_score(pipe, X_train, y_train, cv=cv)
```

O scaler é reajustado só nas dobras de treino de cada rodada.

Hiperparâmetro do passo: `model__alpha`

# O que cada função recebe e retorna

| CHAMADA | RECEBE | RETORNA |
|---|---|---|
| `Ridge(alpha=1.0)` | hiperparâmetros | estimador não treinado |
| `.fit(X, y)` | X: n × p · y: n | o próprio objeto, com coef_ e intercept_ |
| `.predict(X_novo)` | X_novo: m × p | vetor com m previsões |
| `GridSearchCV(pipe, grid, cv, scoring)` | estimador, valores a testar, dobras, métrica | buscador não treinado |
| `.fit(X_train, y_train)` | treino completo | best_params_, best_score_, best_estimator_, cv_results_ |

`scoring="neg_root_mean_squared_error"` → RMSE = -score

# Hiperparâmetros e seus efeitos

| PARÂMETRO | AUMENTAR | REDUZIR |
|---|---|---|
| `alpha` | coeficientes menores; mais zeros no Lasso | aproxima do MQO |
| `l1_ratio` | aproxima do Lasso | aproxima do Ridge |
| `max_iter` | mais passadas; evita ConvergenceWarning | mais rápido; pode não convergir |
| `tol` | para antes, menos preciso | mais preciso, mais lento |
| `random_state` | fixa o sorteio das dobras; garante reprodutibilidade | |

# Quiz rápido

**Q1** No Ridge, aumentar α faz os coeficientes:

a) crescerem b) diminuírem em módulo c) zerarem todos

**Q2** RMSE de treino 12, de validação 41. O diagnóstico é:

a) underfitting b) modelo ideal c) overfitting

**Q3** Qual penalidade produz coeficientes exatamente zero, e por quê?

**Q4** Sem padronizar, o que acontece com a penalidade?

a) depende das unidades b) nada muda c) só o intercepto muda

**Q5** Certo ou errado: padronizar todo o treino e depois rodar a CV.

# Questão de raciocínio

Use os três pontos do slide 7.

$S_{xy} = 3$ $S_{xx} = 2$

a Calcule $\beta_1$ do Ridge com $\alpha = 4$.

b Calcule $\beta_1$ do Lasso com $\alpha = 4$.

c Que $\alpha$ leva cada modelo a $\beta_1 = 0{,}5$?

d O que o item c revela sobre comparar valores de $\alpha$?

CAPÍTULO 08 — LAB

# Escolher alpha por validação cruzada

Notebook novo. load_diabetes: 442 pacientes, 10 variáveis, alvo numérico.


01 Imports
02 Dataset
03 Exploração
04 Treino / teste
05 Pipeline
06 Baseline MQO
07 Ridge
08 Lasso
09 ElasticNet
10 Validação cruzada
11 Melhor alpha
12 Coeficientes
13 Caminho
14 Teste final
15 Interpretação

# Imports, dataset e exploração

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from sklearn.datasets import load_diabetes
from sklearn.model_selection import (train_test_split, KFold,
                                     cross_val_score, GridSearchCV)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import (LinearRegression, Ridge,
                                  Lasso, ElasticNet)
from sklearn.metrics import mean_squared_error, r2_score

SEED = 42
print("sklearn", sklearn.__version__)
X, y = load_diabetes(return_X_y=True, as_frame=True)
print(X.shape)                          # (442, 10)
print(X.corrwith(y).sort_values())  # correlação com o alvo
```

**OBJETIVO**

Ambiente completo e dados carregados

**as_frame=True**

X vira DataFrame com nomes de colunas

**corrwith**

correlação de cada variável com y

**REGISTRE**

a versão do sklearn no relatório

# Divisão, dobras e baseline

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED)

cv = KFold(n_splits=5, shuffle=True, random_state=SEED)
SCORING = "neg_root_mean_squared_error"def make_pipe(model):
    return Pipeline([("scaler", StandardScaler()),
                     ("model", model)])

baseline = make_pipe(LinearRegression())
scores = cross_val_score(baseline, X_train, y_train,
                         cv=cv, scoring=SCORING)
print(f"MQO RMSE-CV = {-scores.mean():.2f} ± {scores.std():.2f}")
```

`test_size=0.2`
20% trancados até o passo 14

`cv`
as mesmas 5 dobras para todos os modelos

`make_pipe`
scaler + modelo; passos "scaler" e "model"

`RESULTADO`
5 scores negativos; –média = RMSE-CV

# Ridge e Lasso com busca de alpha

```
ridge_search = GridSearchCV(
    make_pipe(Ridge()),
    {"model__alpha": np.logspace(-3, 3, 61)},
    cv=cv, scoring=SCORING)
ridge_search.fit(X_train, y_train)

lasso_search = GridSearchCV(
    make_pipe(Lasso(max_iter=50_000)),
    {"model__alpha": np.logspace(-3, 1.5, 46)},
    cv=cv, scoring=SCORING)
lasso_search.fit(X_train, y_train)

print(ridge_search.best_params_, -ridge_search.best_score_)
print(lasso_search.best_params_, -lasso_search.best_score_)
```

`logspace(-3, 3, 61)`
61 alphas de 0,001 a 1000

`grades diferentes`
o Lasso divide o erro por 2n

`custo`
61 × 5 + 1 = 306 treinos de Ridge

`VERIFIQUE`
α* na borda da grade? Amplie.

# ElasticNet e comparação por CV

```
enet_search = GridSearchCV(
    make_pipe(ElasticNet(max_iter=50_000)),
    {"model__alpha": np.logspace(-3, 1.5, 31),
     "model__l1_ratio": [0.1, 0.3, 0.5, 0.7, 0.9]},
    cv=cv, scoring=SCORING)
enet_search.fit(X_train, y_train)

resumo = pd.DataFrame({
    "modelo": ["MQO", "Ridge", "Lasso", "ElasticNet"],
    "RMSE_CV": [-scores.mean(), -ridge_search.best_score_,
                -lasso_search.best_score_,
                -enet_search.best_score_]})
print(resumo.sort_values("RMSE_CV"))
```

**grade 2D**
31 × 5 = 155 combinações

**l1_ratio < 1**
1,0 seria o próprio Lasso

**resumo**
mesmas dobras, métrica e treino

**INTERPRETE**
diferença menor que o desvio entre dobras não é vitória

# Coeficientes e caminho do Lasso

```
baseline.fit(X_train, y_train)
def get(pipe):
    return pipe.named_steps["model"].coef_

coefs = pd.DataFrame({
    "MQO": get(baseline),
    "Ridge": get(ridge_search.best_estimator_),
    "Lasso": get(lasso_search.best_estimator_),
    "ElasticNet": get(enet_search.best_estimator_)},
    index=X.columns)
print(coefs.round(1)); print((coefs == 0).sum())

Xs = StandardScaler().fit_transform(X_train)
alphas = np.logspace(-3, 1.5, 60)
path = [Lasso(alpha=a, max_iter=50_000).fit(Xs, y_train).coef_
        for a in alphas]
plt.semilogx(alphas, path); plt.xlabel("alpha")
plt.ylabel("coeficiente"); plt.show()
```

`best_estimator_`
pipeline já retreinado com α*

`named_steps`
acesso aos passos pelo nome

`coefs == 0`
variáveis eliminadas por modelo

`path`
só visualização; sem validação

# Avaliação final no teste

```
candidatos = {"MQO": baseline, "Ridge": ridge_search,
              "Lasso": lasso_search, "ElasticNet": enet_search}
escolhido = resumo.loc[resumo["RMSE_CV"].idxmin(), "modelo"]
modelo_final = candidatos[escolhido]

y_pred = modelo_final.predict(X_test)   # teste: uma única vez
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)
print(f"{escolhido}: RMSE teste = {rmse:.2f} | R² = {r2:.3f}")
```

`idxmin`
escolha feita só pela CV

`** 0.5`
raiz do MSE = RMSE

`r2_score`
1 perfeito, 0 = prever a média

Depois de ver o teste, não troque de modelo.

# Resultados do experimento

| MQO | Ridge | Lasso | ElasticNet |
|---|---|---|---|
| RMSE-CV | RMSE-CV · α* | RMSE-CV · α* · zeros | RMSE-CV · α* · ρ* |
| preencher após Run All | preencher após Run All | preencher após Run All | preencher após Run All |

| MODELO ESCOLHIDO | RMSE NO TESTE | $R^2$ NO TESTE |
|---|---|---|
| — — — | — — — | — — — |

Campos tracejados: preencher com a saída real de Restart Kernel → Run All · registre a versão do sklearn

# Quatro perguntas de interpretação

01 O ganho sobre o MQO supera o desvio padrão entre dobras?

02 O α* ficou no interior da grade?

03 Quais variáveis o Lasso zerou? Eram correlacionadas com outras?

04 O RMSE de teste ficou próximo do RMSE-CV?

# Erros comuns

| ERRO | CORREÇÃO |
|---|---|
| Regularizar sem padronizar | StandardScaler no Pipeline |
| Padronizar antes da CV | Pipeline reajusta o scaler por dobra |
| Escolher α olhando o teste | Escolher por CV; teste uma vez |
| Mesma grade para Ridge e Lasso | Grades próprias; verificar a borda |
| Ignorar ConvergenceWarning | Aumentar max_iter |
| Zero no Lasso = variável irrelevante | Checar estabilidade e correlações |
| Ordenar score negativo como RMSE | Trocar o sinal antes de comparar |

# Desafio: três penalidades, várias intensidades

1. Curvas de RMSE-CV × α para os três modelos (ElasticNet com ρ = 0,5)
2. Coeficientes zerados em 5 valores de α
3. Regra de um desvio padrão: maior α a 1 dp do mínimo
4. Estabilidade dos zeros do Lasso com 3 sementes

## CRITÉRIOS DE ENTREGA

Notebook que roda em Restart → Run All

Gráficos com eixos, unidades e legenda

Tabela com os números reais obtidos

Texto de 10–15 linhas justificando a escolha

Teste usado apenas no final

# Leitura complementar

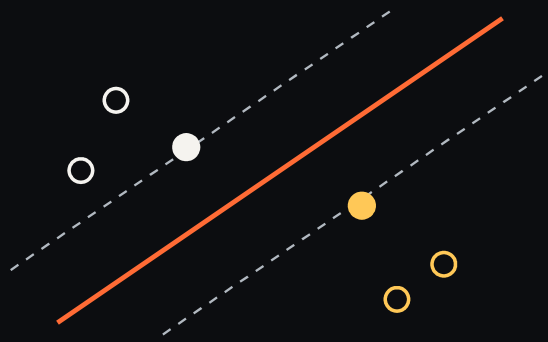
| | |
|---|---|
| PRINCIPAL | James, Witten, Hastie, Tibshirani. An Introduction to Statistical Learning, 2ª ed., cap. 6 |
| AVANÇADO | Hastie, Tibshirani, Friedman. The Elements of Statistical Learning, seção 3.4 |
| ORIGINAIS | Hoerl & Kennard (1970) · Tibshirani (1996) · Zou & Hastie (2005) |
| CÓDIGO | scikit-learn User Guide, Linear Models: funções objetivo de cada classe |

# Síntese

Erro + α · penalidade controla a complexidade

L2 encolhe todos; L1 pode zerar

α por CV dentro do Pipeline; teste uma vez



PRÓXIMA — 7.2

## SVM e o truque do kernel

Margem máxima = minimizar $\|w\|^2$, a penalidade L2 de hoje